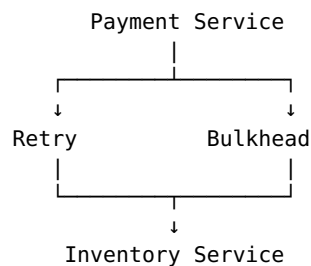


Retry + Bulkhead - Complete Interview Notes

3. RETRY + 4. BULKHEAD — Complete Interview Notes

These two patterns are commonly used together with **Circuit Breaker** in Spring Boot microservices.



3. RETRY

What is Retry?

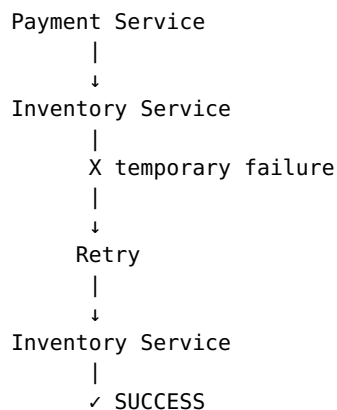
Retry automatically calls a failed operation again a limited number of times.

It is useful for **temporary/transient failures**.

Examples:

Network temporarily unavailable
Connection timeout
HTTP 503
Temporary service unavailability

Example:

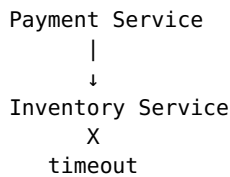


Simple interview definition

Retry means automatically attempting a failed operation again when the failure may be temporary.

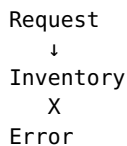
1. Why do we need Retry?

Suppose:

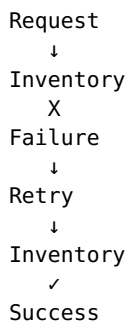


Sometimes the failure is temporary.

Without Retry:



With Retry:



2. Add Resilience4j dependency

For your Gradle Spring Boot project:

```
dependencies {
    implementation 'org.springframework.cloud:spring-cloud-starter-
circuitbreaker-resilience4j'
    implementation 'org.springframework.boot:spring-boot-starter-
actuator'
}
```

If you are using the Resilience4j annotations directly, the relevant Resilience4j Spring Boot integration also needs to be present through your chosen starter.

3. Retry configuration

```
resilience4j:
  retry:
    instances:
      inventoryService:
        # Total attempts = 3
        max-attempts: 3
        # Wait 500ms before retry
        wait-duration: 500ms
        # Enable exponential backoff
        enable-exponential-backoff: true
        # Backoff multiplier
```

`exponential-backoff-multiplier: 2`

The retry sequence becomes approximately:

```
Attempt 1
  ↓
FAIL
  ↓
500 ms
  ↓
Attempt 2
  ↓
FAIL
  ↓
1000 ms
  ↓
Attempt 3
  ↓
SUCCESS
```

So:

```
500ms
1 second
2 seconds
...
```

depending on the configured backoff behavior.

4. Java @Retry

```
@Service
public class InventoryClientService {

    @Retry(
        name = "inventoryService",
        fallbackMethod = "fallback"
    )
    public Inventory checkStock(String skuId) {
        return inventoryClient.getStock(skuId);
    }

    public Inventory fallback(
        String skuId,
        Throwable throwable) {
        return Inventory.unavailable(skuId);
    }
}
```

Flow:

```
checkStock()
  ↓
Attempt 1
  ↓
FAIL
  ↓
Attempt 2
  ↓
FAIL
  ↓
Attempt 3
  ↓
SUCCESS
```

If all attempts fail:

Attempt 1 → FAIL

Attempt 2 → FAIL
Attempt 3 → FAIL
↓
fallback()

5. What should NOT be retried?

This is very important in interviews.

GET

Usually safer:

GET /users/100

If it fails because of a temporary network problem:

GET
↓
FAIL
↓
RETRY

Generally reasonable.

Payment POST

Be careful:

POST /payments

Imagine:

Payment Service
↓
Bank
↓
₹5,000 payment SUCCESS
X
Response lost

Your service doesn't know whether the payment succeeded.

If you blindly retry:

POST /payments
↓
Bank
↓
₹5,000 SUCCESS AGAIN

Potential duplicate payment.

Therefore:

Non-idempotent operation
+
Retry
↓
Potential duplicate side effect

6. How to safely retry payment?

Use an **Idempotency Key**.

Request:

POST /payments
Idempotency-Key: PAY-100001

First request:

PAY-100001
↓
Payment processed
↓
SUCCESS

Retry:

PAY-100001
↓
Already processed?
↓
YES
↓
Return previous result

Therefore:

Retry
+
Idempotency Key
=
Safe retry for many non-idempotent operations

The exact idempotency implementation belongs in the business/payment layer, not merely in the retry configuration.

7. Exponential Backoff

Bad:

Failure
↓
Retry immediately
↓
Failure
↓
Retry immediately
↓
Failure

If 10,000 clients do this:

10,000 requests
↓
10,000 retries
↓
10,000 more retries
↓
Service overloaded

Instead:

Attempt 1
↓
500ms
↓
Attempt 2
↓
1s
↓

Attempt 3

This gives the downstream service time to recover.

8. What is Jitter?

At large scale, even exponential backoff can synchronize clients.

Suppose everyone waits exactly:

500ms

Then:

1000 clients

↓

wait 500ms

↓

1000 requests arrive together

Jitter adds randomness to the delay.

For example:

Client 1 → 480ms

Client 2 → 530ms

Client 3 → 610ms

Client 4 → 455ms

This spreads retries over time.

Interview answer

“Jitter adds randomness to retry backoff so that many clients don’t retry at exactly the same time, reducing the thundering-herd problem.”

9. Retry + Circuit Breaker

Don’t think:

Retry forever

Instead:

Retry

↓

limited attempts

↓

Circuit Breaker

↓

if dependency remains unhealthy

↓

OPEN

↓

fallback

For example:

Request

↓

Retry

↓

Attempt 1 → FAIL

↓

Attempt 2 → FAIL

↓
Attempt 3 → FAIL
↓
Circuit Breaker detects failures
↓
OPEN
↓
Future calls fail fast

The exact decorator ordering should be chosen based on which failures you want each component to observe.

4. BULKHEAD

What is Bulkhead?

Bulkhead isolates resources so that one failing/slow dependency cannot consume all resources of your application.

The name comes from a ship:

Room A	Room B	Room C
Flood ✕	Normal ✓	Normal ✓

If Room A floods, the entire ship doesn't necessarily flood.

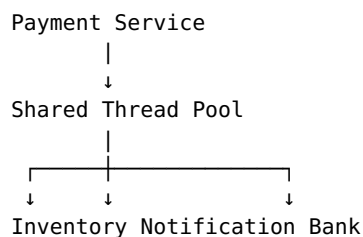
Same idea in microservices.

10. Problem without Bulkhead

Suppose Payment Service calls:

Inventory Service
Notification Service
Bank Service

And all use the same resources:



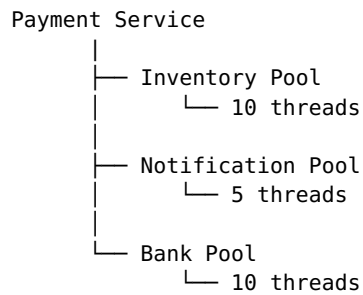
Inventory becomes extremely slow:

Inventory
↓
Slow
↓
Consumes threads
↓
All threads occupied
↓
Notification requests blocked
↓
Bank requests blocked

One dependency can affect unrelated dependencies.

11. Bulkhead solution

Separate resources:



If Inventory becomes slow:

Inventory Pool



FULL ✕

But:

Notification Pool



AVAILABLE ✓

Therefore Notification can continue working.

12. Thread Pool Bulkhead

Your example:

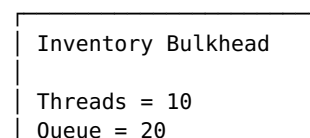
```
@Bulkhead(  
    name = "inventoryService",  
    type = Bulkhead.Type.THREADPOOL  
)  
public CompletableFuture<Inventory> checkStock(  
    String skuId) {  
    return CompletableFuture.supplyAsync(  
        () -> inventoryClient.getStock(skuId)  
    );  
}
```

Configuration:

```
resilience4j:  
  thread-pool-bulkhead:  
    instances:  
      inventoryService:  
        max-thread-pool-size: 10  
        core-thread-pool-size: 5  
        queue-capacity: 20
```

Conceptually:

Inventory Requests



If all 10 threads are busy, additional requests can wait in the configured queue, and once capacity is exhausted, calls are rejected rather than consuming unlimited resources.

13. Semaphore Bulkhead

There is another type:

```
@Bulkhead(  
    name = "inventoryService",  
    type = Bulkhead.Type.SEMAPHORE  
)
```

Conceptually:

Maximum concurrent calls = 10

Request 1 ✓
Request 2 ✓
Request 3 ✓
...
Request 10 ✓
Request 11 ✕

It limits **concurrent executions**, rather than creating a dedicated thread pool.

14. Semaphore vs Thread Pool

	Semaphore	Thread Pool
Main purpose	Limit concurrency	Isolate execution resources
Creates separate threads	No	Yes
Resource isolation	Lower	Stronger
Async execution	Not by itself	Yes
Complexity	Simpler	More complex
Good for	Fast/simple calls	Slow/blocking external calls

Interview answer

"I use a Semaphore Bulkhead when I mainly need to limit concurrent calls and don't need a separate execution pool. I use a Thread-Pool Bulkhead when I need stronger isolation of execution resources, particularly for blocking or slow downstream operations."

15. How do you decide pool size?

Don't simply say:

Always use 10 threads.

Size it based on:

Downstream latency
Expected throughput

Application CPU
Connection pool size
SLA
Concurrency
Timeout
Traffic patterns

For example, suppose:

Expected throughput = 100 requests/sec
Average downstream latency = 100ms

A rough concurrency estimate using Little's Law:

$$\begin{aligned}\text{Concurrency} &\approx \text{Throughput} \times \text{Latency} \\ &= 100 \times 0.1 \\ &= 10 \text{ concurrent requests}\end{aligned}$$

So you might start around:

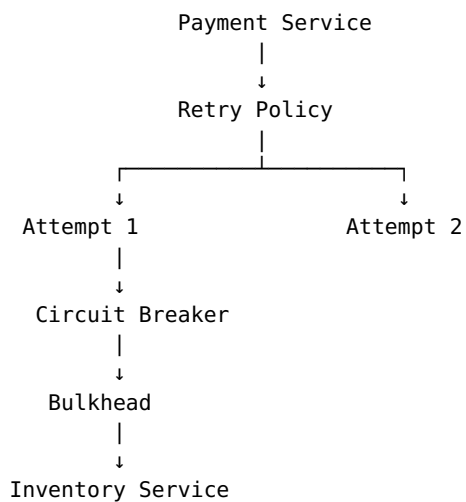
10 concurrent calls

and validate using real production metrics.

This is a **starting estimate**, not a universal production setting.

16. Retry + Bulkhead + Circuit Breaker

Now combine everything:



If Inventory has a temporary failure:

Retry → try again

If Inventory is continuously failing:

Circuit Breaker → OPEN

If Inventory is consuming too many resources:

Bulkhead → limit concurrency

If Inventory doesn't respond quickly:

Timeout → stop waiting

If the operation can safely degrade:

Fallback → return alternative response

17. Complete configuration

For your interview/demo project:

```
resilience4j:

# =====
# CIRCUIT BREAKER
# =====
circuitbreaker:
  instances:
    inventoryService:
      failure-rate-threshold: 50
      minimum-number-of-calls: 10
      sliding-window-size: 10
      wait-duration-in-open-state: 10s
      permitted-number-of-calls-in-half-open-state: 3
      automatic-transition-from-open-to-half-open-enabled: true

# =====
# RETRY
# =====
retry:
  instances:
    inventoryService:
      max-attempts: 3
      wait-duration: 500ms
      enable-exponential-backoff: true
      exponential-backoff-multiplier: 2

# =====
# THREAD POOL BULKHEAD
# =====
thread-pool-bulkhead:
  instances:
    inventoryService:
      max-thread-pool-size: 10
      core-thread-pool-size: 5
      queue-capacity: 20
```

18. Complete Java example

```
@Service
public class InventoryClientService {

    @Retry(
        name = "inventoryService",
        fallbackMethod = "fallback"
    )
    @CircuitBreaker(
        name = "inventoryService",
        fallbackMethod = "fallback"
    )
    @Bulkhead(
        name = "inventoryService",
        type = Bulkhead.Type.SEMAPHORE
    )
    public Inventory checkStock(String skuId) {
        System.out.println(
            "Calling Inventory Service: " + skuId
        );
        return inventoryClient.getStock(skuId);
    }

    public Inventory fallback(
        String skuId,
```

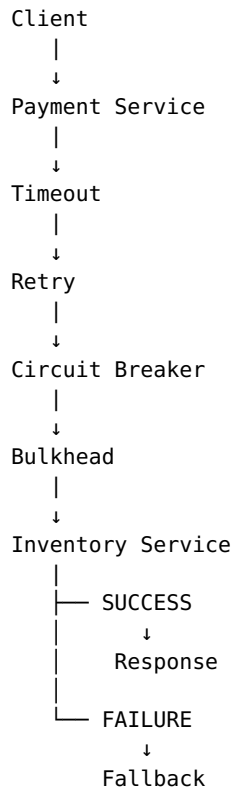
```

        Throwable throwable) {
    System.out.println(
        "Fallback triggered for: " + skuId
    );
    return Inventory.unavailable(skuId);
}
}

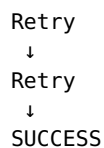
```

For a **Thread-Pool Bulkhead**, use an asynchronous return type such as `CompletableFuture` and configure `thread-pool-bulkhead` appropriately.

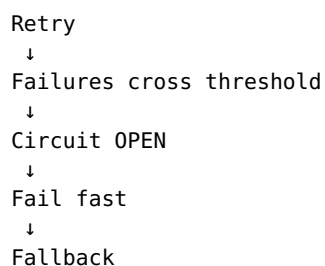
19. Real production flow



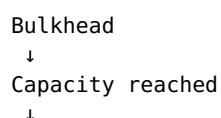
If the service is temporarily failing:



If it's continuously failing:



If too many concurrent calls arrive:



20. Interview questions

0-2 years

Q: What is Retry?

Retry automatically attempts a failed operation again when the failure may be temporary.

Q: What is Bulkhead?

Bulkhead isolates resources so one slow dependency doesn't consume resources needed by other operations.

2-4 years

Q: Would you retry a payment POST?

Not blindly. A payment operation can have a side effect even when the response is lost. I would use an idempotency key and retry only according to a controlled policy.

Q: Why exponential backoff?

It increases the delay between retries, giving the downstream service time to recover and reducing pressure during an outage.

Q: Why Bulkhead?

To prevent one slow dependency from exhausting all application resources and affecting other healthy dependencies.

4-7 years

Q: What is Jitter?

Jitter introduces randomness into retry delays so many clients don't retry simultaneously, reducing the thundering-herd problem.

Q: How do you size a Bulkhead?

I look at expected throughput, downstream latency, concurrency, timeout, connection-pool capacity, CPU and the dependency SLA. I use these to establish an initial limit and then tune it using production metrics.

Q: Semaphore vs Thread-Pool Bulkhead?

Semaphore limits concurrent executions without creating a dedicated execution pool. Thread-Pool Bulkhead provides a separate execution pool and is useful when stronger execution-resource isolation is required, especially for blocking/slow calls.

Q: How do Retry and Circuit Breaker work together?

Retry handles transient failures with bounded attempts and backoff. Circuit Breaker handles sustained dependency failures by stopping calls temporarily. I make sure retry is bounded and doesn't amplify traffic during an outage, and I monitor both retry volume and circuit state.

The easiest way to remember all 5

TIMEOUT

↓

"Don't wait forever"

RETRY

↓

"Try again"

CIRCUIT BREAKER

↓

"Stop calling a failing service"

BULKHEAD

↓

"Limit the damage"

FALLBACK

↓

"Give a graceful response"

For a **5+ year Spring Boot interview**, this combination — **Retry + Circuit Breaker + Bulkhead + Timeout + Idempotency + monitoring** — is much more important than simply knowing the annotations.